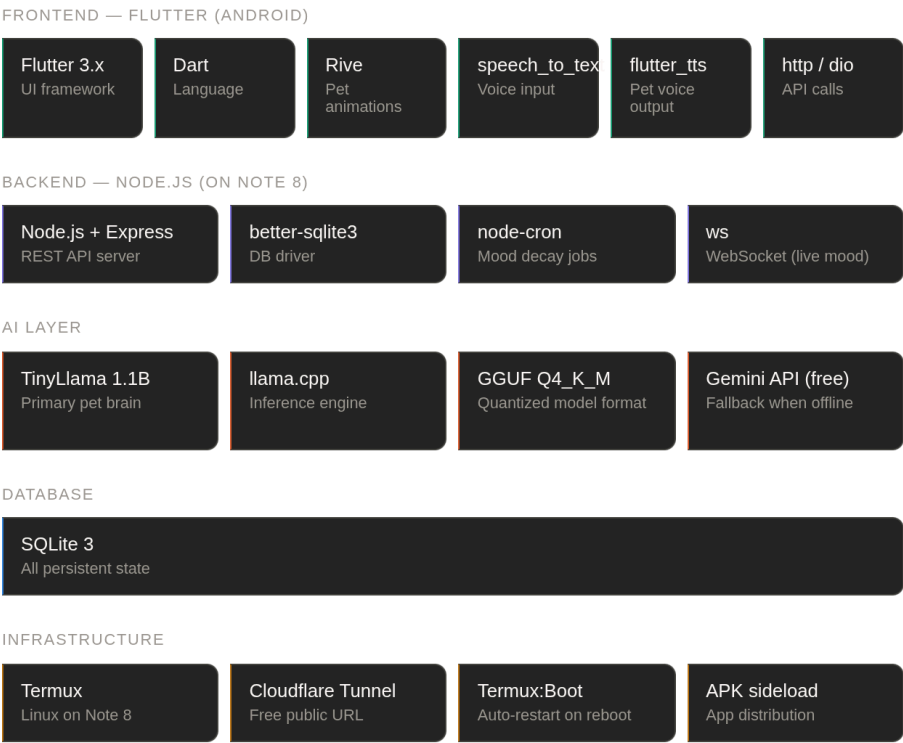


Mochi

Application Description

Mochi is a family-shared AI companion pet app. One creature lives on a shared server and grows wiser, more expressive, and more emotionally nuanced the more your family interacts with it. Each family member has their own identity and affection score with the pet, but the pet itself is singular — a shared family bond. It speaks in short, warm sentences, remembers moments, reacts to moods, and visibly evolves through life stages. It is not a smart assistant. It is a companion.

Tech Stack

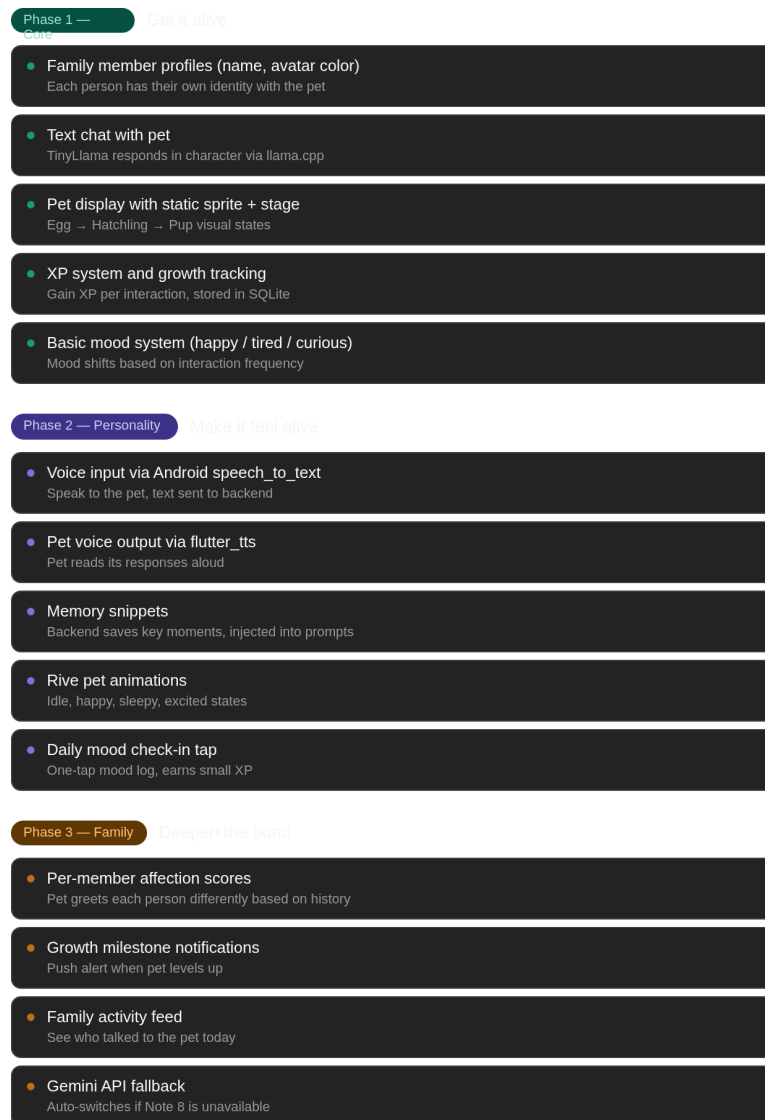


This diagram shows the full technology stack of Mochi, organized by system layers. The frontend is built using Flutter, which handles the user interface, animations, and input/output features like voice interaction. The backend runs on Node.js with Express, managing API logic, mood processing, and scheduled tasks.

The AI layer uses TinyLlama running locally via llama.cpp for fast, private inference, with a Gemini API fallback when the local model is unavailable. SQLite serves as the lightweight database for storing all persistent data such as pet state, interactions, and memories.

Infrastructure components like Termux, Cloudflare Tunnel, and Termux:Boot allow the system to run continuously on a mobile device while remaining accessible over the internet.

MVP Features



This diagram outlines the phased development plan of Mochi. Each phase builds on the previous one to gradually enhance the pet's capabilities.

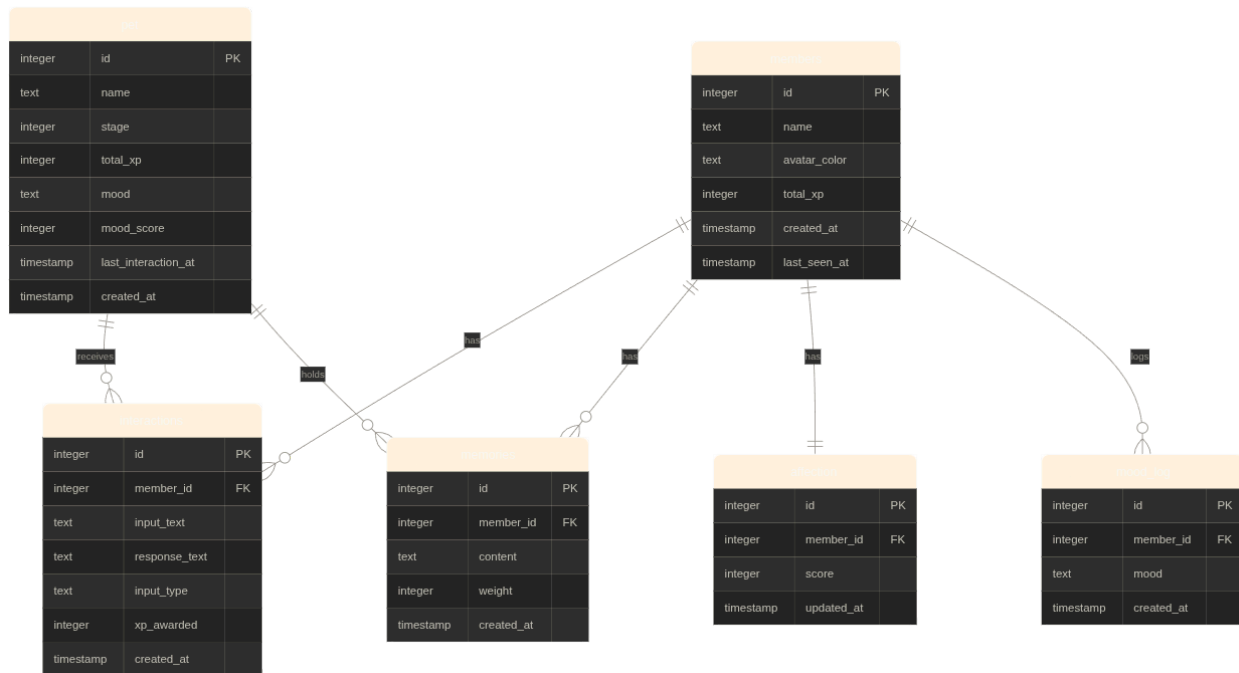
- **Phase 1 (Core):** Establishes the foundation, including user profiles, chat interaction, pet display, and a basic growth and mood system.
- **Phase 2 (Personality):** Adds emotional depth through voice interaction, memory retention, animations, and daily engagement features.
- **Phase 3 (Family):** Expands the shared experience by introducing affection tracking, activity feeds, and notifications, making the pet feel like a true family entity.

This phased approach ensures that the app remains stable while progressively increasing complexity and emotional richness.

Device Setup - Samsung Galaxy Note 8

Step	What to do	Detail
1	Install Termux	From F-Droid (not Play Store — the Play version is outdated)
2	Install Termux:Boot	Auto-restarts your server after reboots
3	Install Node.js in Termux	<code>pkg install nodejs</code>
4	Install llama.cpp	Compile from source in Termux — ARM-optimized build
5	Download TinyLlama GGUF	<code>tinyllama-1.1b-chat-v1.0.Q4_K_M.gguf</code> (~700MB)
6	Install Cloudflare Tunnel	<code>cloudflared</code> binary for ARM, set up free tunnel
7	Keep plugged in always	USB-C charger, cap battery at 85% via Bixby Routine
8	Disable screen timeout	Keep Termux sessions alive
9	Set up startup script	Termux:Boot launches Node.js + llama.cpp server on boot

Database Tables and Columns



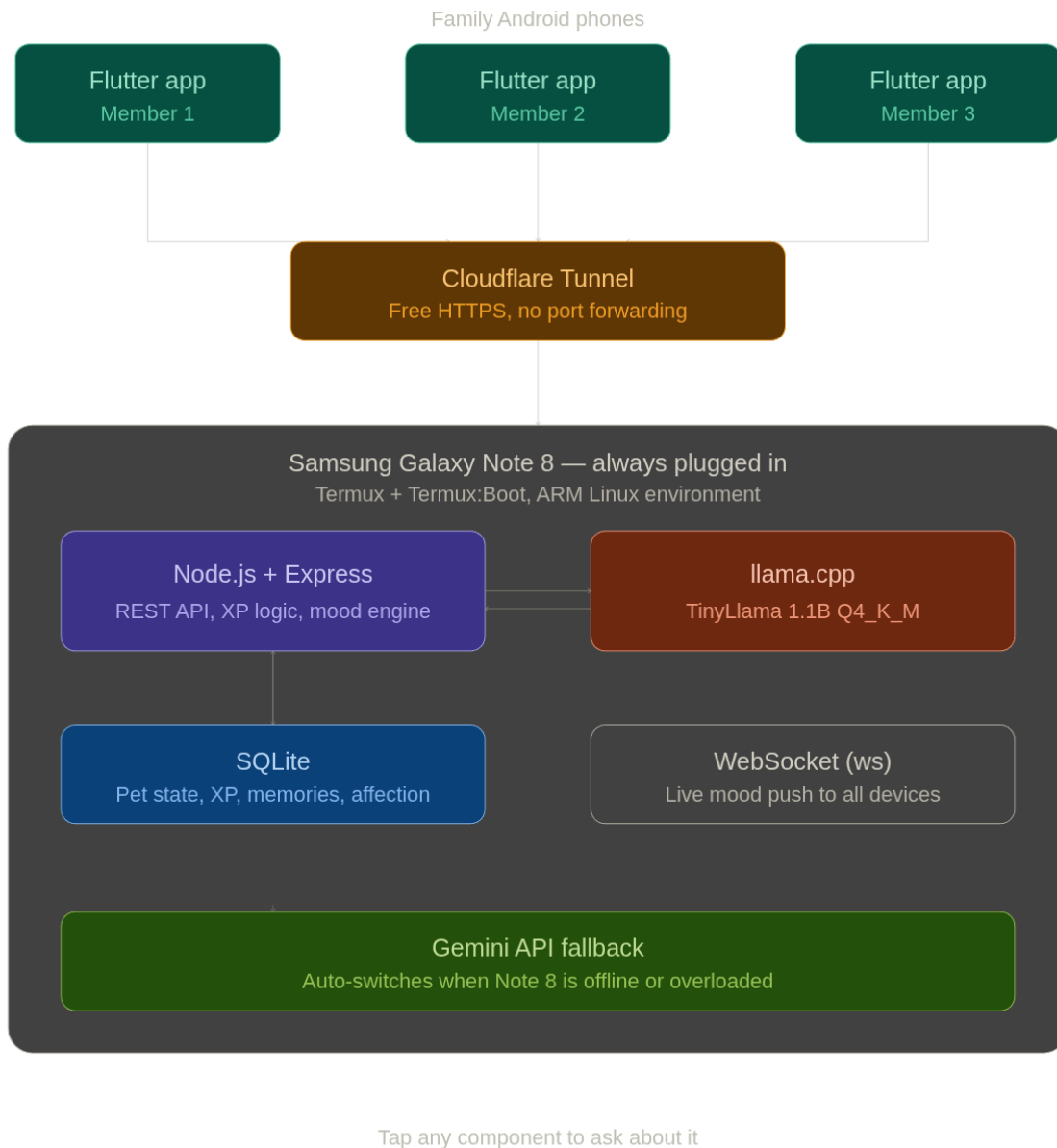
Here's what each table stores in plain terms. **members** holds each family member's profile and total XP. **pet** is a single-row table — one pet for the whole family — tracking its stage, mood, and XP. **interactions** is every conversation turn, logging who said what, the pet's response, and how much XP was awarded. **memories** stores short extracted summaries of meaningful moments, injected into the AI prompt to simulate long-term recall. **affection** tracks each member's personal closeness score with the pet. **mood_log** records daily check-in taps.

The diagram visualizes the relationships between core entities in the system. The **pet** table acts as the central node, representing the single shared companion.

Each **member** interacts with the pet through the **interactions** table, which logs conversations and XP gains. The **memories** table stores summarized meaningful events that influence future AI responses.

The **affection** table links members to the pet, tracking emotional closeness, while **mood_log** captures daily mood inputs. These relationships ensure that both individual and shared experiences shape the pet's behavior over time.

Architectural Design



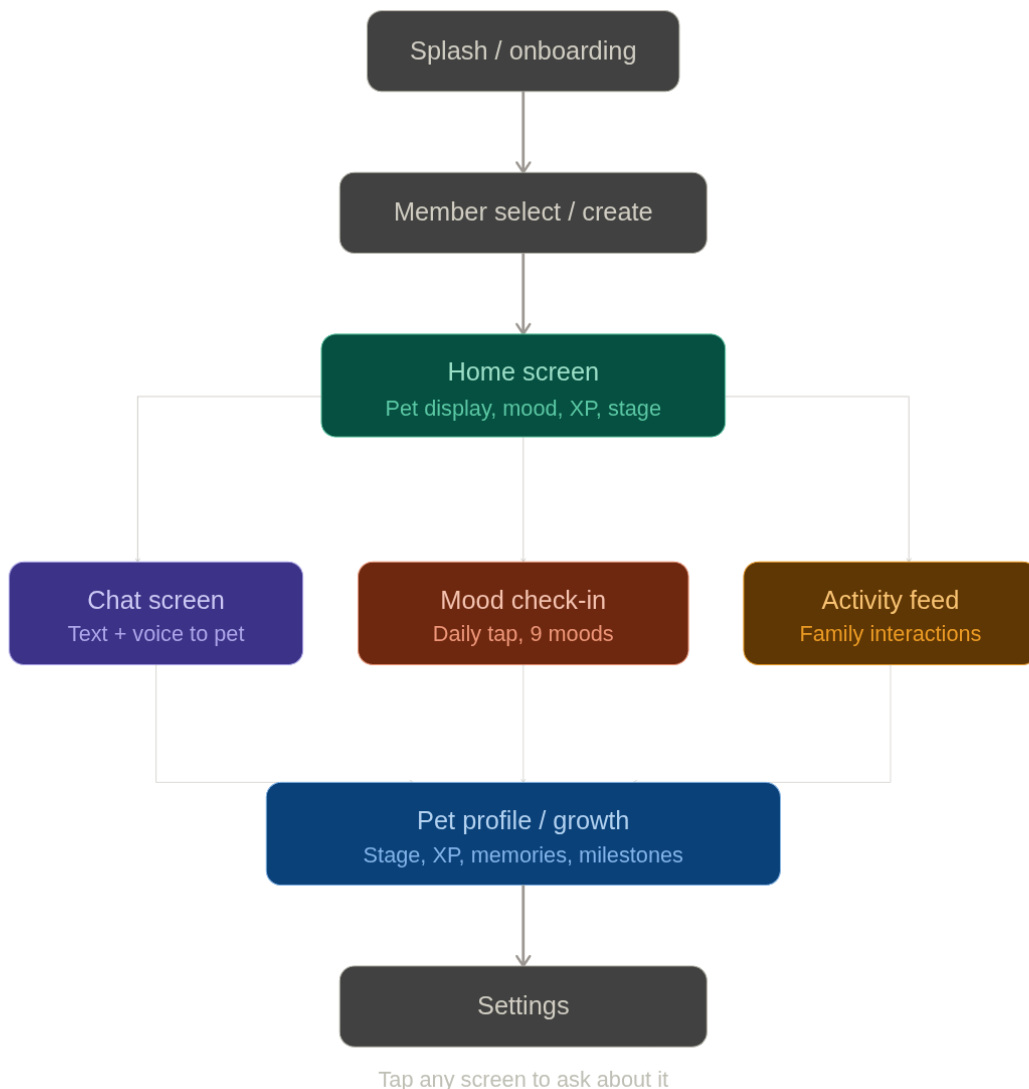
The architecture flows in one clean path: Flutter apps on your family's phones talk over HTTPS to a Cloudflare Tunnel, which routes to your Note 8. Node.js handles all API logic — it reads from SQLite, builds the prompt with pet state and memory snippets, calls llama.cpp for TinyLlama's response, writes the result back, and pushes mood updates to all connected phones via WebSocket. If the Note 8 is unreachable, the Flutter app automatically falls back to Gemini API so the pet is never completely dark.

This diagram illustrates how all components of Mochi communicate in real time. Multiple Flutter apps (family members) connect through HTTPS to a Cloudflare Tunnel, which securely exposes the local server running on the Samsung Galaxy Note 8.

Inside the device, Node.js acts as the central orchestrator. It processes requests, retrieves data from SQLite, constructs AI prompts, and sends them to TinyLlama via llama.cpp. The generated responses are then returned to users and broadcast to all devices using WebSocket for live updates.

If the local server becomes unavailable, the system automatically switches to the Gemini API, ensuring uninterrupted interaction.

Screen Navigation Map



All Screen Details

Home

Home screen

The main screen everyone lands on after selecting their profile. The pet lives here — animated, reacting, and always showing its current state. This is the heart of the app.

- display** Animated pet sprite — changes pose based on current mood (idle, happy bounce, droopy tired, shaking scared, etc.)
- display** Pet name and current growth stage badge (Egg / Hatchling / Pup / Companion / Wise Elder)
- display** XP progress bar showing how far to the next stage
- display** Current mood label with color-coded pill (e.g. green for Happy, blue for Sad)
- display** Greeting message from the pet — a short line from TinyLlama personalized to the member
- action** Tap the pet to trigger a surprise reaction — random short response + animation
- action** Pet petting gesture (long press) — earns +2 XP, plays happy animation
- nav** Bottom nav bar: Home / Chat / Mood / Feed / Profile
- logic** WebSocket listener — mood and XP update live when another family member interacts
- logic** Fallback indicator — small banner if Note 8 is offline and Gemini is active

Chat

Chat screen

Where the real conversation happens. The pet responds through TinyLlama, reads replies aloud via TTS, and the chat history scrolls like a messenger thread.

- display** Chat bubble thread — member messages on the right, Mochi replies on the left with a small pet avatar
- display** Typing indicator (animated dots) while TinyLlama generates a response
- display** Pet mood tag visible above the chat — updates if mood shifts during conversation
- action** Text input field with send button — sends message to /chat API
- action** Mic button — holds to record voice, uses Android speech_to_text, sends transcribed text
- action** Speaker toggle — mute/unmute TTS voice output for the pet's replies
- action** Tap any pet reply to have it re-read aloud
- logic** Response is trimmed server-side to 2–3 sentences max before display
- logic** +10 XP awarded per chat message sent, reflected immediately in home screen
- logic** Chat history is local-only — not stored on server, only summaries are saved as memories

Mood check-in

Mood check-in screen

A once-per-day ritual. The family member picks how they're feeling, and the pet reacts to it — shifting its own mood slightly in response. Simple, warm, and earns XP.

- display** 3×3 grid of the 9 mood tiles — each with a color, label, and simple shape icon
- display** Selected mood highlights with a border and scale-up animation
- display** Pet's reaction message after selection — a short warm response acknowledging the mood
- display** "Already checked in today" state — shows today's mood and locks the grid
- action** Tap a mood tile to submit — calls /mood-checkin, awards +5 XP
- action** Confirm button after selection before submitting
- logic** One check-in per member per calendar day — enforced client-side with local timestamp
- logic** Member's mood is logged to mood_log table, influences pet mood_score
- logic** Pet mood shifts: if most family members check in as Tired or Sad, pet mood drifts toward Tired

Activity feed

Activity feed

A shared family timeline. Everyone can see who talked to Mochi, what mood they checked in as, and when the pet leveled up. Creates a sense of shared ownership.

- display** Chronological feed of events: chats, mood check-ins, XP milestones, stage-ups
- display** Each entry shows: member avatar color + name, event type, time ago (e.g. "Dad chatted · 2h ago")
- display** Stage-up events shown as full-width celebration cards with the new stage name
- display** Today's check-in moods shown as a small row of colored dots at the top
- action** Pull to refresh — fetches latest interactions from server
- logic** Feed data pulled from interactions + mood_log tables, sorted by created_at DESC
- logic** Chat message content is not shown — only "chatted with Mochi" to preserve some privacy

Pet Profile

Pet profile / growth screen

A deeper look at Mochi's life so far — all 5 growth stages, XP history, memories the pet holds, and each family member's affection score.

- display** 5-stage growth timeline — current stage highlighted, past stages faded, future stages locked
- display** Total XP count and XP needed for next stage
- display** Memory cards — short text snippets of things the pet "remembers" (pulled from memories table)
- display** Affection leaderboard — family members ranked by affection score with the pet
- display** Pet's birthday (created_at date) and age in days
- action** Tap a memory card to see its full text
- action** Tap a stage to see what it unlocks

Onboarding

Splash / onboarding

Shown only on first launch. Introduces Mochi, lets the first user name the pet, and creates the first family member profile.

- display** Petling logo and app name on launch
- display** 3-step intro: meet your pet, how it grows, family sharing
- action** Name the pet — text field, saved to pet.name on server
- action** Create first member profile — enter name, pick avatar color
- action** Enter the server URL (Cloudflare tunnel URL) — saved to local shared preferences
- logic** On subsequent launches, goes straight to member select screen
- logic** Connectivity check — pings /pet endpoint, shows error if server unreachable

Member select / create

Shown every time the app opens after onboarding. Each family member picks their profile before entering — keeping affection scores and XP separate.

- display** List of existing family member cards — avatar color circle + name + total XP
- action** Tap a profile to select it and enter the app
- action** Add new member button — name + color picker, calls POST /members

Settings

Settings screen

Practical controls for managing the app and server connection.

- action** Change server URL — update Cloudflare tunnel address if it changes
- action** TTS voice selector — choose from available Android TTS voices
- action** TTS speed slider — slow / normal / fast
- action** Notification toggle — enable/disable stage-up push alerts
- action** Switch member — returns to member select screen
- display** Server status indicator — green (online) / red (offline / using Gemini fallback)
- display** App version and pet's current server stats (uptime, last interaction)

Moods

The 9 moods — full reference

Each mood has a color, a behavior effect on the pet's replies, and an animation state for the sprite.

 Happy Bouncy idle · warm replies · bright green	 Sad Slow droop · gentle soft replies · muted blue	 Angry Short terse replies · red · shaking
 Normal Default state · neutral replies · gray	 Tired Slow blink · yawning anim · sleepy purple	 Confused Head tilt · "?" replies · amber
 Laughing Bouncing · playful jokes · teal	 Hungry Belly rub anim · food hints · coral	 Scared Shivering · timid replies · pink

How moods change

- logic** Mood starts as Normal on first launch
- logic** Frequent chat interactions push mood toward Happy or Laughing
- logic** No interactions for 12+ hours drifts mood toward Tired or Hungry
- logic** Family check-ins heavily influence mood — if most members log Sad, pet shifts toward Sad
- logic** TinyLlama system prompt is updated with current mood before every reply
- logic** node-cron job runs every hour on the server to recalculate mood from recent data
- logic** Mood change is broadcast to all connected phones via WebSocket immediately

This diagram represents the user flow throughout the app. Users start at the onboarding or splash screen, where they create or select a profile. From there, they enter the home screen, which serves as the central hub.

The home screen branches into three main interactions:

- Chat (communicating with the pet)
- Mood check-in (daily emotional input)
- Activity feed (shared family interactions)

All paths eventually connect to the pet profile screen, where users can view growth, memories, and progress. The settings screen is accessible for configuration and system management.

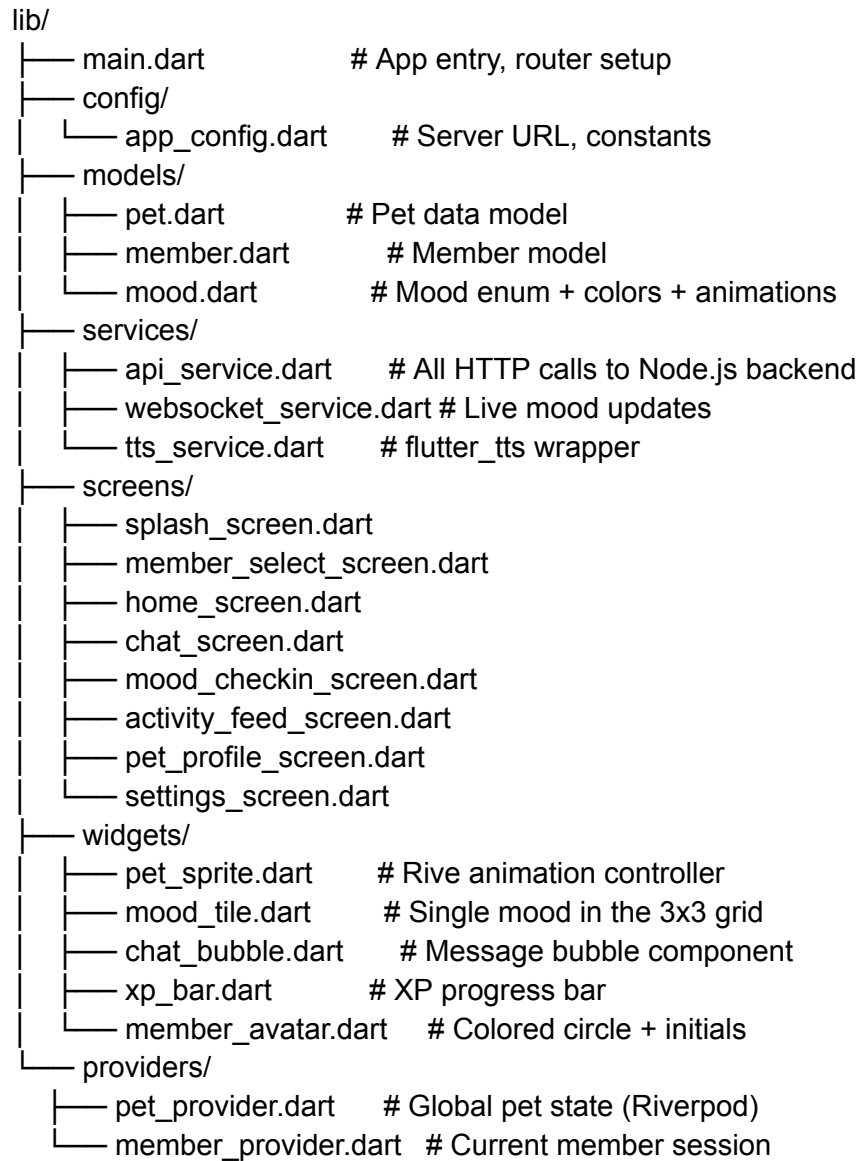
This structure ensures a simple, loop-based navigation centered around the pet experience.

This diagram defines the emotional states of the pet and how each mood affects its behavior and presentation. Each mood includes a color, animation style, and response tone, allowing users to quickly understand the pet's emotional state.

The system dynamically adjusts the pet's mood based on interaction patterns. Frequent engagement leads to positive moods like Happy or Laughing, while inactivity or negative inputs shift the pet toward Tired or Sad states.

This design creates a feedback loop where user behavior directly influences the pet's personality, reinforcing emotional connection.

Flutter File Structure



This diagram shows how the Flutter project is organized to maintain clean architecture and scalability. The codebase is divided into clear layers:

- **models/** define the data structures
- **services/** handle API calls, WebSocket communication, and TTS
- **screens/** represent UI pages
- **widgets/** contain reusable UI components
- **providers/** manage global state using Riverpod

This separation of concerns improves maintainability, makes features easier to extend, and supports collaborative development.